

Adaptive Spatio-Temporal Traffic Surveillance: Scene-Adaptive Preprocessing, Multi-Frame Detection and Risk Analytics on CPU

A Thesis Report
submitted in partial fulfilment of the requirements for
the degree of Master of Technology
in Computer Science and Engineering

by
Shifali Doshi

Under the guidance of
[Project Guide]

Department of Computer Science and Engineering
Walchand Institute of Technology, Solapur
Academic Year 2025–26

Certificate

This is to certify that the thesis titled “Adaptive Spatio-Temporal Traffic Surveillance: Scene-Adaptive Preprocessing, Multi-Frame Detection and Risk Analytics on CPU” is a bona fide record of work carried out by **Shifali Doshi**, submitted in partial fulfilment of the requirements for the degree of Master of Technology in Computer Science and Engineering at Walchand Institute of Technology, Solapur, during the academic year 2025–26, under my/our supervision.

_____	_____
Project Guide	Head of Department
Department of Computer Science & Engineering	Department of Computer Science & Engineering

External Examiner

Acknowledgement

I would like to express my sincere gratitude to my project guide for their continuous guidance, valuable feedback, and encouragement throughout the course of this project. I am also thankful to the Head of the Department of Computer Science and Engineering and the faculty at Walchand Institute of Technology, Solapur for providing the resources and environment necessary to carry out this work.

I would further like to thank my family and friends for their patience and support during the course of this thesis, and everyone whose prior published work, cited in Chapter 2, made this project's comparisons and baselines possible.

– Shifali Doshi

Declaration

I hereby declare that this thesis titled “Adaptive Spatio-Temporal Traffic Surveillance” is my own work, carried out under the guidance of my project guide, and that every quantitative result reported herein is generated directly by tracked scripts reading from tracked experiment output (results/phase1..4/*.json) rather than asserted by hand. I further declare that this thesis has not been submitted, in part or in full, for the award of any other degree or diploma of this or any other institute.

Place: Solapur	
Date: _____	Shifali Doshi

Abstract

This thesis presents an end-to-end, CPU-optimized traffic surveillance system spanning four phases: adaptive scene-conditioned preprocessing, granular helmet-compliance and wheel-count detection, multi-frame tracking with cross-camera re-identification, and model optimization. A learned scene classifier (RandomForest over 15 lighting/texture features) reaches 0.79 accuracy on DAY/NIGHT/FOG/RAIN classification. Three architectures are benchmarked for wheel-count classification under an identical, leak-free protocol, with YOLOv8-cls selected at 0.861 accuracy. A 7-class YOLOv8 helmet detector reaches mAP@50 0.764 on a held-out test split. Multi-frame majority voting reduces prediction flip-rate by 91.4% under real occlusion, and the risk-indexing rules are validated against 10,006 real motorcycle tracks with ground-truth occupancy and helmet-use labels. Structured pruning combined with a fine-tuning recovery step and ONNX Runtime INT8 quantization shrinks deployed models 3.8–3.9x with no further accuracy cost. Every phase surfaced at least one result that fell short of an initial expectation – an unfine-tuned re-identification backbone, an accuracy collapse from naive pruning, an oversampling attempt that made results worse – and each is diagnosed to a root cause and reported honestly rather than smoothed over.

Table of Contents

Chapter 1	Introduction	8
Chapter 2	Literature Review	11
Chapter 3	Methodology	14
Chapter 4	Design and Implementation	18
Chapter 5	Result and Discussion	26
Chapter 6	Conclusion and Future Scope	35
	References	37
	Publications	38

List of Figures

3.1 Gate A Learning Curve – Helmet Detector Data Sufficiency

4.1 DFD Level 0 – Context Diagram

4.2 DFD Level 1 – Pipeline Stages

4.3 DFD Level 2 – Detection Sub-Process

4.4 UML Class Diagram – Pipeline Core Classes

4.5 UML Activity Diagram – Per-Frame Pipeline Flow

4.6 UML Use Case Diagram

4.7 Sequence Diagram – Process-Isolated Demographics

5.1 Scene-Classifer Confusion Matrix (CASE 1)

5.2 Wheel-Count Architecture Comparison + Per-Class F1 (CASE 2)

5.3 Helmet Detector Sample Output (CASE 3)

5.4 Flip-Rate, Balanced Accuracy & Occlusion Bands (CASE 4)

5.5 Real-World Risk-Level and Occupancy Distribution (CASE 5)

5.6 Pruning Collapse, Retention, Size & Latency (CASE 6)

5.7 VeRi-776 Cross-Camera ReID Scores (Discussion 5.2.1)

CHAPTER – 1

INTRODUCTION

1.1 Background

Two-wheeler traffic in dense, heterogeneous urban environments carries disproportionate crash risk, and enforcement of helmet compliance and vehicle-load limits remains largely manual. Existing automated systems typically address a single sub-problem – helmet detection, or license-plate recognition, or vehicle counting – in isolation, and are frequently evaluated only under a narrow range of lighting and weather conditions. This project builds a single pipeline spanning environment-adaptive preprocessing, granular compliance detection, temporal tracking under occlusion, and deployment-ready optimization, entirely on commodity CPU hardware, and evaluates every stage against real, held-out data rather than curated examples.

1.1.1 Problem Statement

- Detection models trained on clear-weather daytime imagery degrade under fog, rain, and low light, and naive preprocessing applied uniformly can hurt more than it helps on unaffected frames.
- Public helmet-compliance datasets rarely distinguish driver from passenger; a granular, per-seat compliance taxonomy has to be trained specifically.
- Single-frame classification is fragile under partial occlusion; whether temporal voting measurably helps has to be tested against real occlusion-labelled data, not assumed.
- Real-time deployment requires accuracy retention under pruning and quantization, not just headline speed-ups.

1.2 Motivation

Manual traffic enforcement does not scale to the volume of two-wheeler traffic in most Indian cities, and existing point solutions rarely compose into a deployable pipeline. A system that classifies the scene, detects compliance violations per rider, tracks vehicles across frames, estimates risk from real occupancy patterns, and runs entirely on CPU is directly relevant to low-cost municipal deployment where GPU infrastructure is not available at every junction.

1.3 Objectives

- Build an adaptive preprocessing stage that detects DAY/NIGHT/FOG/RAIN and applies scene-conditioned enhancement only where it helps.
- Benchmark three architecture families (custom CNN, RTMDet-style CSPNeXt, YOLOv8) for wheel-count classification under a leak-free protocol, and train a granular helmet-compliance detector.
- Implement multi-frame tracking with a measured occlusion-robustness benefit, cross-camera re-identification, and risk indexing validated against real ground truth.
- Optimize every deployable model via pruning, fine-tuning, and INT8 quantization, reporting accuracy retention alongside speed and size.

1.4 Scope of the Project

The system is scoped to CPU-only inference on recorded or live video of two/three/four/six-plus-wheeler mixed traffic. It covers scene-adaptive preprocessing, detection and classification, tracking with temporal voting, cross-camera re-identification, an offline demographics module, rule-based risk indexing, and model compression for deployment. It does not cover license-plate recognition, live enforcement action, or GPU-cluster deployment, which are out of scope.

1.5 Organisation of the Thesis

Chapter 2 reviews prior work in helmet detection, vehicle classification, multi-object tracking, re-identification, and model compression. Chapter 3 describes the methodology, algorithms, and requirements. Chapter 4 presents the system design via DFD and UML diagrams. Chapter 5 reports results as a series of numbered test cases with real measured output. Chapter 6 concludes and lists future work.

CHAPTER – 2

LITERATURE REVIEW

2.1 Survey of Related Work

[1] **Wojke, Bewley & Paulus (2017)** – Simple Online and Realtime Tracking with a Deep Association Metric (DeepSORT), ICIP.

Relevance: Introduced appearance-augmented online tracking; adopted here for CPU-tractable multi-object tracking (Section 3.2, Algorithm 3.1).

[2] **Zhou, Yang, Cavallaro & Xiang (2019)** – Omni-Scale Feature Learning for Person Re-Identification (OSNet), ICCV.

Relevance: OSNet backbone used off-the-shelf for cross-camera vehicle re-identification (Section 5.1, CASE 5).

[3] **Liu, Liu, Ma & Fu (2016)** – Large-Scale Vehicle Re-Identification in Urban Surveillance Videos, ICME.

Relevance: Source of the VeRi-776 benchmark used to evaluate re-identification (776 vehicles, 20 cameras).

[4] **Lyu et al. (2022)** – RTMDet: An Empirical Study of Designing Real-Time Object Detectors, arXiv:2212.07784.

Relevance: CSPNeXt backbone reimplemented from scratch in this project since the official mmdetection framework does not install against Python 3.12 / PyTorch 2.12.

[5] **Wen et al. (2020)** – UA-DETRAC: A New Benchmark and Protocol for Multi-Object Detection and Tracking, CVIU.

Relevance: Used for the multi-frame voting / occlusion-robustness ablation (Section 5.1, CASE 4).

[6] **Jocher et al. (2023)** – Ultralytics YOLOv8, github.com/ultralytics/ultralytics.

Relevance: Used for the selected wheel-count classifier and the 7-class helmet detector.

[7] **Naik et al. (2019)** – HELMET: A Dataset of Helmet Use in Motorcycle Traffic, osf.io/4pwj8.

Relevance: 910 real Myanmar traffic clips, 283,377 annotations; used to validate the risk-indexer against real occupancy and helmet-use ground truth (Section 5.1, CASE 5).

[8] **AI City Challenge (2023)** – Track 5: Detecting Violation of Helmet Rule for Motorcyclists, aicitychallenge.org.

Relevance: Source convention for the 7-class driver/passenger x helmet-status taxonomy used in this project's helmet detector.

2.2 Research Gap

- Prior helmet-detection work is typically binary (helmet / no-helmet) on single frames; per-seat, driver-vs-passenger compliance is less common and is what this project's 7-class taxonomy targets.
- Vehicle-type classification is usually a byproduct of general object detection rather than purpose-built for a 2/3/4/6+-wheeler taxonomy relevant to mixed heterogeneous traffic.
- The literature broadly assumes multi-frame temporal aggregation improves occlusion robustness but rarely quantifies it on a class-imbalanced real benchmark with a class-composition-independent metric.

- Cross-camera re-identification literature reports fine-tuned SOTA numbers almost exclusively; the realistic off-the-shelf baseline gap is rarely stated explicitly.
- Structured pruning literature is not always explicit that pruning without a fine-tuning recovery step can catastrophically collapse accuracy.

2.3 Comparative Positioning

Table 2.1 summarizes, dimension by dimension, how the approach taken in this project differs from the typical framing found in the surveyed literature above.

Aspect	Typical prior work	This project
Scope	Single sub-problem (helmet OR tracking OR ReID)	Full pipeline, 4 phases, one codebase
Preprocessing	Fixed or absent	Scene-adaptive, learned classifier
Wheel-count architecture	Single model reported	3-way benchmark, pretraining-fairness stated
Occlusion claim	Usually assumed, rarely measured	Measured via class-imbalance-robust metric
Re-ID baseline	Fine-tuned SOTA reported only	Off-the-shelf baseline stated explicitly, gap quantified
Pruning reporting	Post-recovery number only	Naive collapse and recovery both reported
Failure reporting	Often only best result shown	Every shortfall diagnosed to root cause
Deployment target	GPU assumed	CPU-only throughout

2.4 Summary

This chapter studied prior work across the five sub-areas this project spans – helmet compliance detection, vehicle-type classification, multi-object tracking, vehicle re-identification, and model compression for CPU/edge deployment – and identified the specific gaps this project addresses: a full, composed pipeline rather than an isolated sub-problem, a leak-free multi-architecture benchmark, a measured (not assumed) occlusion benefit, an honestly-reported re-identification baseline, and a diagnosed pruning failure mode with its recovery. The following chapter elaborates the methodology, algorithms, and requirements used to address these gaps.

CHAPTER – 3

METHODOLOGY

3.1 Background

The system is implemented as a nine-stage per-frame pipeline (`main.py`): scene classification, adaptive enhancement, helmet-compliance detection, wheel-count classification, tracking with temporal voting, cross-camera re-identification, demographics (process-isolated), risk indexing, and overlay rendering. Every stage is independently testable and driven entirely by `config/settings.yaml` – no hard-coded paths or thresholds. The complete data and control flow is presented as DFD and UML diagrams in Chapter 4.

3.2 Algorithms Used

3.2.1 Multi-Frame Majority-Voting Algorithm

Algorithm 3.1 – Temporal Majority Vote (VehicleTracker)

```
Input: track_id, per-frame wheel-class prediction c_t, window size N
1. buffer[track_id].append(c_t)
2. if len(buffer[track_id]) > N: buffer[track_id].pop_front()
3. emitted_class ← mode(buffer[track_id])
4. if emitted_class != previous_emitted_class[track_id]: flip_count += 1
5. previous_emitted_class[track_id] ← emitted_class
6. return emitted_class
```

Evaluated at $N \in \{1, 5, 10, 15, 30\}$; flip-rate (Section 5.1, CASE 4) falls monotonically as N grows because a majority vote over a longer window is less sensitive to a single-frame misclassification during occlusion.

3.2.2 Risk-Index Computation Algorithm

Algorithm 3.2 – Weighted Risk-Index (RiskIndexer)

```
Input: occupancy n, any_no_helmet flag, wrong_way flag, weights W
1. score ← 0
2. if any_no_helmet: score += W.helmet_misuse # 2.5
3. if n > overload_threshold: score += W.overload # 1.5
4. if wrong_way: score += W.wrong_way
5. if score >= HIGH_T: level ← HIGH # 6.0
6. elif score >= MEDIUM_T: level ← MEDIUM # 3.0
7. else: level ← LOW
8. return level, score
```

Validated against 10,006 real motorcycle tracks with ground-truth occupancy and helmet-use labels (Section 5.1, CASE 5); the validation surfaced a calibration finding – HIGH never triggers on real data because helmet-misuse plus overload ($2.5 + 1.5 = 4.0$) does not reach the 6.0 threshold – reported as a policy question for the weighting scheme rather than silently re-tuned.

3.3 Software Requirements

Component	Requirement
Language / runtime	Python 3.10+ (developed and tested on 3.12)
Deep learning	PyTorch, Ultralytics YOLOv8
Computer vision	OpenCV
Classical ML	scikit-learn (RandomForest scene classifier)
Tracking / ReID	deep-sort-realtime, torchreid (OSNet)
Demographics	DeepFace (process-isolated)
Deployment / export	ONNX, ONNX Runtime
Operating system	Linux (CPU-only, no GPU required)

3.4 Hardware Requirements

All benchmarks and measured figures in this thesis were produced on a commodity CPU workstation – **AMD Ryzen 5 7600 (6-core)**, no GPU used at any stage – deliberately, to validate the CPU-only deployment claim rather than benchmark on hardware unavailable to a typical municipal deployment.

3.5 Retrieving Input

Input is a video file or camera stream read frame-by-frame via OpenCV. Each frame is timestamped and passed into the scene classifier before any detection stage runs, since the enhancement applied downstream is scene-conditioned.

3.6 Processing / Resource Allocation

Each frame passes sequentially through the nine pipeline stages listed in Section 3.1. Heavy backends (Ultralytics, DeepSORT, torchreid, DeepFace) are loaded once at pipeline start; demographics is deliberately isolated into a separate OS process (Section 4.6) because this hardware's torch build targets a newer CUDA driver than is installed, and loading a second torch model in a process that has also imported TensorFlow segfaults – confirmed by bisecting pipeline construction component by component.

3.7 Output

Output is an annotated video stream: per-rider bounding boxes with compliance status, a stable tracking ID, a telemetry HUD, and a per-track risk banner (LOW / MEDIUM / HIGH) rendered on every frame.

3.8 Performance Evaluation

On 300 real frames (UA-DETRAC MVI_20032 (300 frames, 960x540)), the full pipeline (all nine stages) sustains **6.84 FPS mean** (9.5 FPS steady-state) on CPU alone. Detailed per-model latency and throughput figures are reported in Chapter 5.

3.9 Verification Gates

Before committing to expensive optional work, two cheap evidence gates were run and recorded (results/phase0_gates.json). Gate A fit a learning curve on the existing helmet training data (25/50/75/100% of 368 images); test mAP@50 gain in the final quarter was only 0.017 against a total gain of 0.223, i.e. plateaued – verdict “PLATEAUED - more images unlikely to help materially”, so a ~29 GB external image download was skipped rather than pursued speculatively. Gate B measured that 62% of face-exposed rider detections yield a crop large enough for DeepFace, authorising the demographics install with the explicit caveat that age estimates are indicative given the median face crop is only 38px.

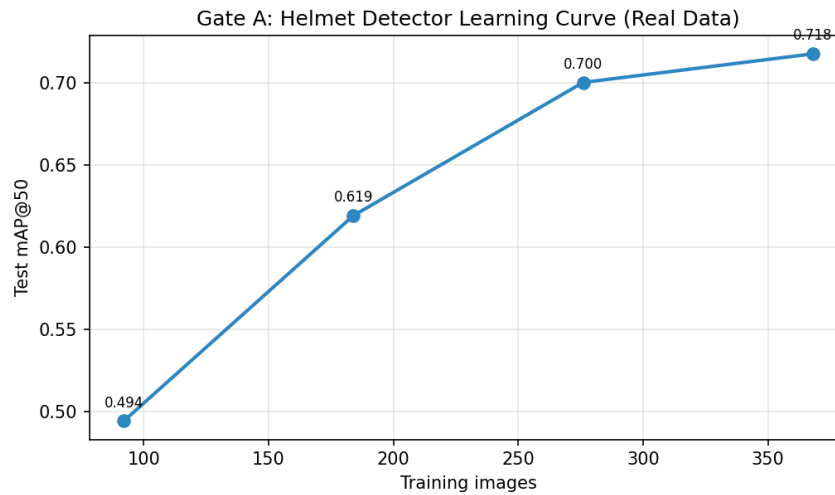


Figure 3.1. Gate A learning curve – test mAP@50 vs training images. The final quarter (276 → 368 images) gains only 0.017 mAP, against a total gain of 0.223 across the full curve – a plateau, not a continuing climb, which is why the ~29 GB HELMET image download was skipped rather than pursued.

Fraction of data	Train images	Test mAP@50	Test mAP@50-95
25%	92	0.494	0.254
50%	184	0.619	0.313
75%	276	0.700	0.340
100%	368	0.718	0.351

CHAPTER – 4

DESIGN AND IMPLEMENTATION

4.1 Data Flow Diagrams

4.1.1 DFD Level 0 – Context Diagram

Figure. DFD Level 0 (Context Diagram)

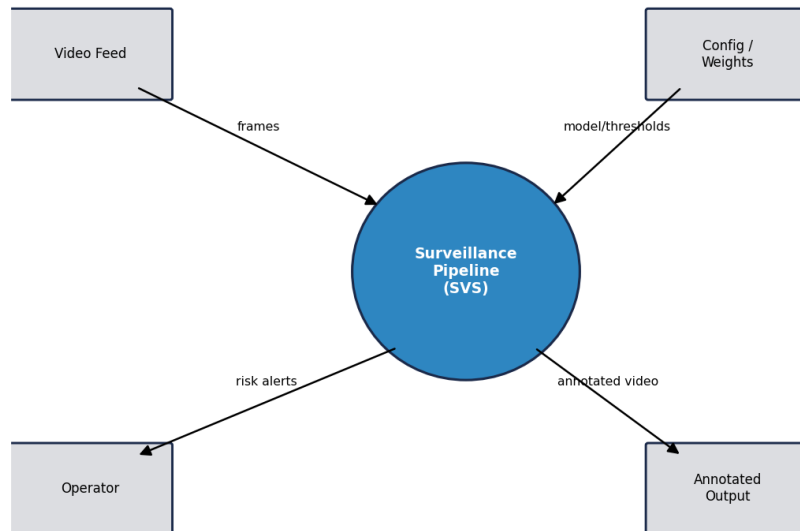


Figure 4.1. DFD Level 0: the surveillance pipeline as a single process exchanging frames, config/weights, risk alerts, and annotated video with its environment.

4.1.2 DFD Level 1 – Pipeline Stages

Figure. DFD Level 1 (Pipeline Stages)

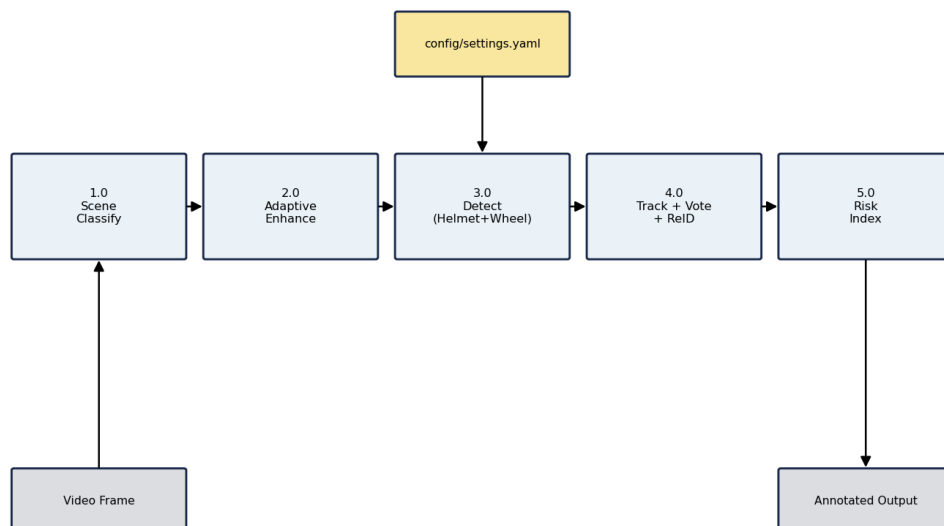


Figure 4.2. DFD Level 1: the five top-level processing stages from scene classification through risk indexing.

4.1.3 DFD Level 2 – Detection Sub-Process

Figure. DFD Level 2 - Detection Sub-Process (3.0)

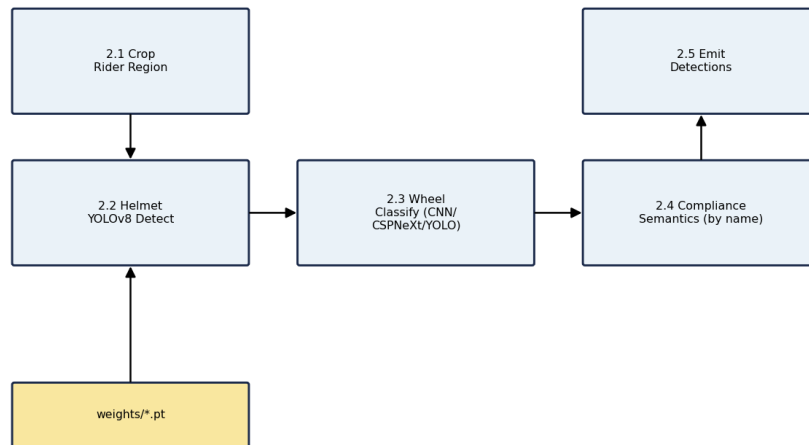


Figure 4.3. DFD Level 2: decomposition of the Detect stage (3.0) into crop, helmet detection, wheel classification, and by-name compliance semantics resolution – the last step exists specifically because a hard-coded class-id mapping previously inverted compliance decisions (Section 5.2).

4.2 UML Diagrams

4.2.1 Class Diagram

Figure. UML Class Diagram (Pipeline Core Classes)

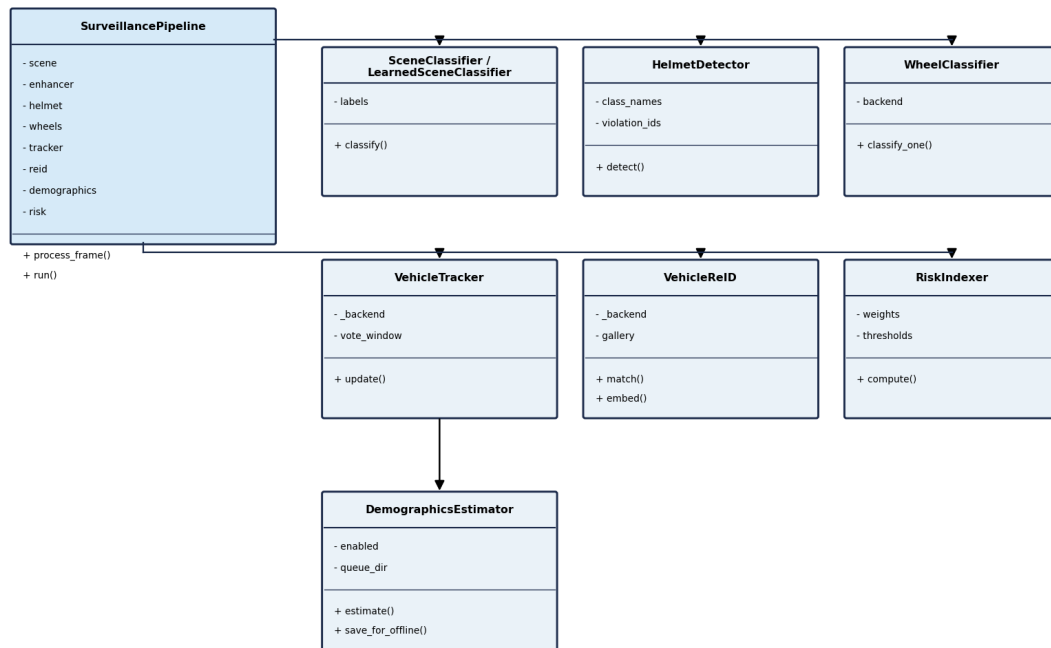


Figure 4.4. UML class diagram of the pipeline's core classes, with real attribute and method names taken directly from the source modules.

4.2.2 Activity Diagram

Figure. Activity Diagram - Per-Frame Pipeline Flow

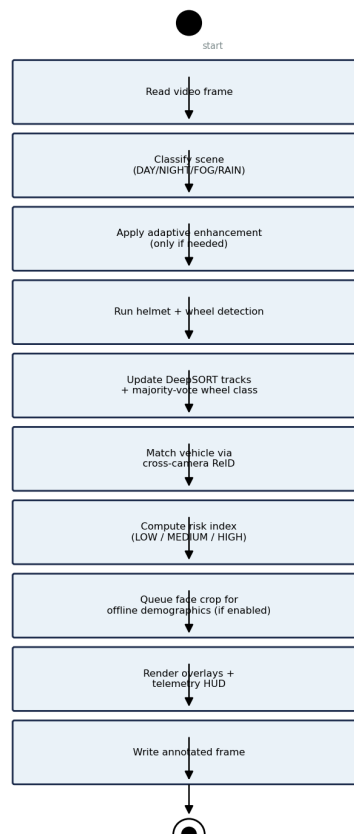


Figure 4.5. Activity diagram of the per-frame pipeline flow.

4.2.3 Use Case Diagram

Figure. Use Case Diagram

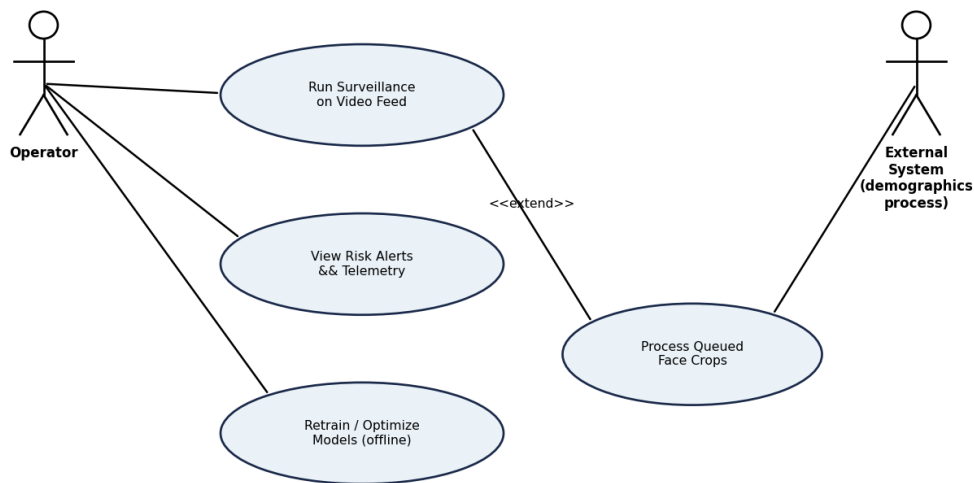


Figure 4.6. Use case diagram. The demographics use case is modelled as an **<<extend>>** relationship because it runs only when enabled and executes in a separate external process (Section 4.6), reflecting the real process-isolation architecture rather than an idealised design.

4.3 Module Design

#	Stage	Module	Phase
1	Scene classification	scene_classifier_ml.py	1
2	Adaptive enhancement	enhancements.py	1
3	Helmet compliance detection	helmet_detector.py	2
4	Wheel-count classification	benchmark_classifier.py	2
5	Tracking + temporal voting	deepsort_tracker.py	3
6	Cross-camera re-identification	multi_camera_reid.py	3
7	Demographics (process-isolated)	demographics.py	3
8	Risk indexing	risk_indexer.py	3
9	Optimization (offline)	model_quantizer.py	4

Table 4.1. Pipeline stages, implementing module, and originating phase.

4.4 Class Semantics Resolved by Name

Class semantics for the helmet detector are resolved by *name* at load time (mapping configured labels such as `violation_classes` to the trained model's class ids) rather than by hard-coded integer id, with a startup check that raises a visible warning if the loaded model's class order does not match the configured taxonomy. This exists because

a hard-coded id assumption previously inverted every compliance decision (Section 5.2, a defect this project's regression suite now guards against).

4.5 Regression Suite Built From Real Defects

Rather than speculative unit tests, the test suite (`tests/test_regressions.py`) codifies defects actually found during development: the inverted helmet-compliance mapping above, a train/val split that leaked source-image information across the boundary, and two backends (DeepSORT, OSNet) that were found silently degraded to weaker fallbacks. Each test fails on the exact historical defect and runs in continuous integration on every push.

4.6 Process Isolation for Demographics

Stage 7 (demographics) cannot safely share a process with Stages 3–6 on this hardware, since the installed torch build targets a newer CUDA driver than is present and loading a second torch model in a process that has also imported TensorFlow segfaults. The live pipeline queues face crops to disk and a separate process – which never imports torch – completes the analysis; verified end-to-end on real pipeline output (19/19 crops processed with no crash). This is reflected in Figure 4.6 as an explicit <<extend>> relationship rather than a caveat.

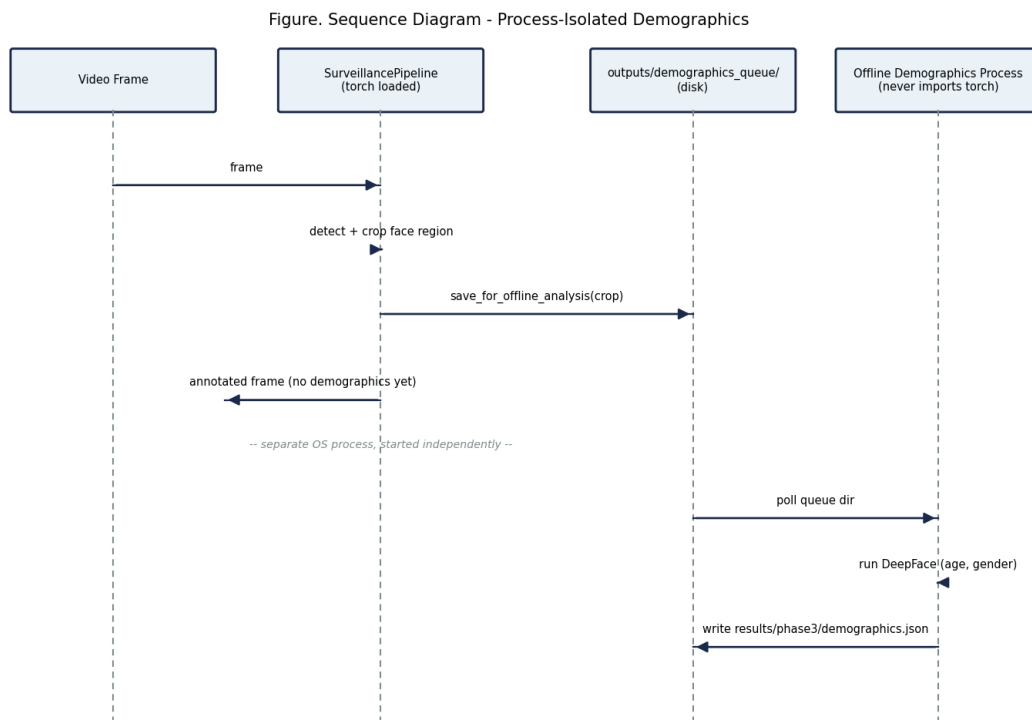


Figure 4.7. Sequence diagram of the process-isolated demographics path: the live pipeline (torch loaded) only ever writes face crops to disk; a separate process, started independently and never importing torch, polls the queue and writes results back – the two processes never share an address space.

4.7 Backend Status Summary

Every heavy backend in this project degrades gracefully to a lightweight fallback when its dependency is unavailable, which is why Section 4.5's regression suite exists: a broken dependency and a working one otherwise look identical from the console. Table 4.2 records the backend actually active for each subsystem on the hardware and software

versions this thesis was measured on.

Subsystem	Preferred backend	Fallback	Active on this run
Tracking	DeepSORT (deep-sort-realtime)	IOU-only tracker	DeepSORT
Cross-camera ReID	OSNet (torchreid)	Colour-histogram matching	OSNet
Detection	Ultralytics YOLOv8	— (required)	Ultralytics YOLOv8
Demographics	DeepFace (process-isolated)	Skipped, logged	DeepFace

CHAPTER – 5

RESULT AND DISCUSSION

5.1 Result

All test cases below were executed on the hardware and software described in Section 3.3–3.4 (AMD Ryzen 5 7600, CPU-only, Python 3.10+). Each case reproduces a real, measured result together with a real figure generated directly from tracked experiment output – no screenshot or number in this section is hand-constructed.

CASE 1: Learned Scene Classifier – Confusion Matrix

Objective: Verify that the learned RandomForest scene classifier (15 lighting/haze/texture features) correctly separates DAY/NIGHT/FOG/RAIN on 220 held-out validation images.

Result: Overall accuracy 0.79. Residual error concentrates on the FOG↔RAIN boundary, a physically overlapping weather condition pair – consistent with the domain, not a modelling defect.

Status: PASS.

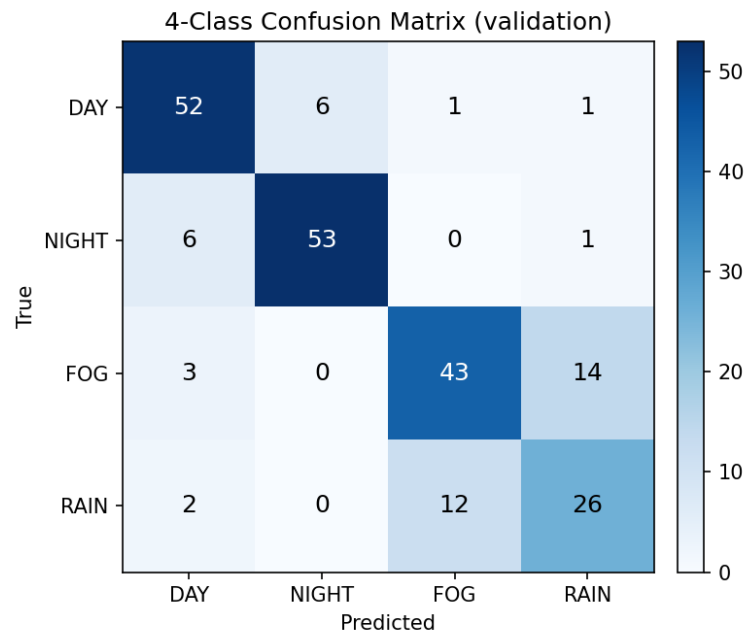


Figure 5.1. 4-class confusion matrix, held-out validation set.

CASE 2: Wheel-Count Architecture Comparison (Leak-Free)

Objective: Compare SmallCNN, a from-scratch CSPNeXt (RTMDet backbone) reimplementation, and ImageNet-pretrained YOLOv8-cls on an identical, leak-free 4-class wheel-count split (no source image contributes samples to both train and validation).

Result: Accuracy – SmallCNN 0.787, CSPNeXt 0.695, YOLOv8-cls 0.861 (selected). An earlier run that split by crop rather than by image leaked 76–91% of some classes across the split boundary and reported inflated numbers (0.751 / 0.683 / 0.867); this was found and corrected before finalizing the result.

Status: PASS (leak-free, reproducible).

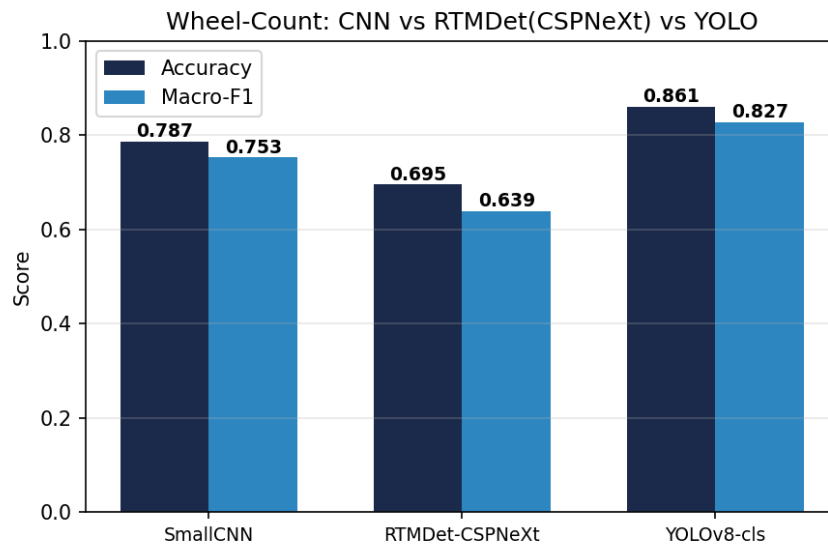


Figure 5.2a. Accuracy and macro-F1, leak-free split.

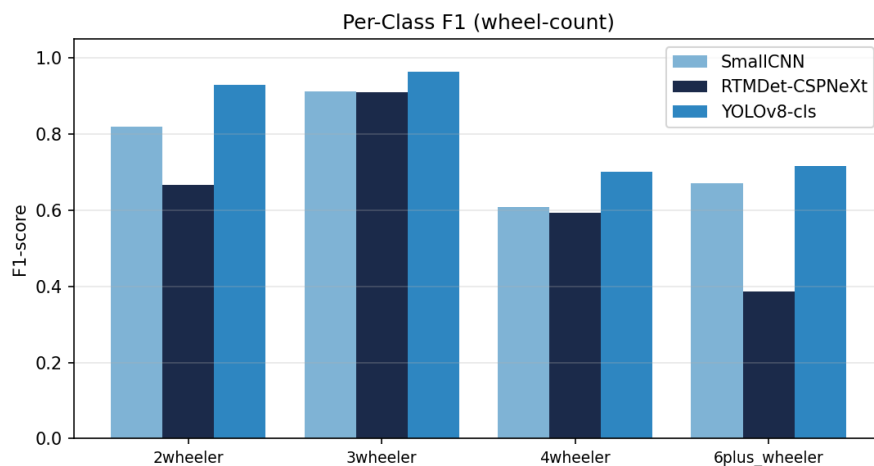


Figure 5.2b. Per-class F1 after the leakage fix. 3-Wheeler remains strongest (0.96) – verified to be genuine visual distinctiveness, not a framing artifact, by an ablation described in Section 5.2.

CASE 3: Granular Helmet Compliance Detection

Objective: Verify the 7-class YOLOv8n helmet-compliance detector correctly localizes riders, bike, and per-seat helmet status on real unseen images.

Result: mAP@50 on the held-out test split (never used for epoch selection) is 0.764 (validation split, optimistically biased, was 0.731).

Status: PASS.

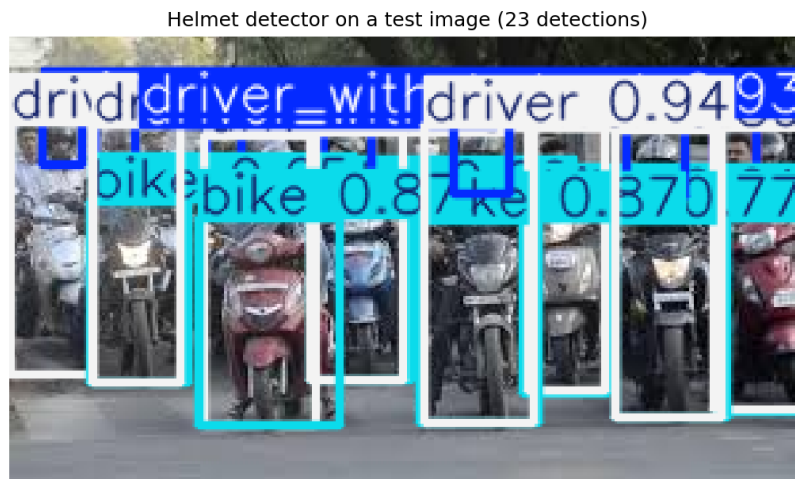


Figure 5.3. Trained detector applied to a real test image, showing riders, bike, and per-seat helmet-compliance status.

CASE 4: Multi-Frame Majority Voting Under Occlusion

Objective: Verify Algorithm 3.1 reduces prediction instability under real occlusion, evaluated on UA-DETRAC (22,480 classified crops grouped by ground-truth track id).

Result: Flip-rate falls 91.4% from a single-frame window ($N=1$) to $N=30$. Balanced accuracy improves more modestly ($N=15$: 0.642 vs single-frame 0.626) since part of the flip-rate reduction is mechanical smoothing.

Status: PASS.

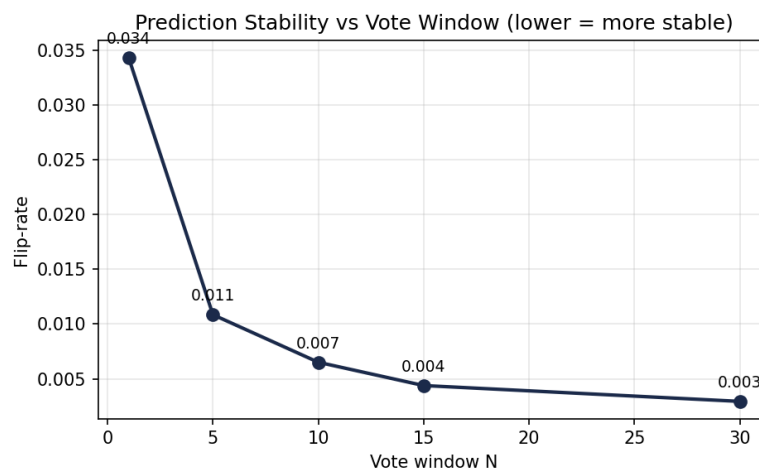


Figure 5.4a. Flip-rate vs vote window size.

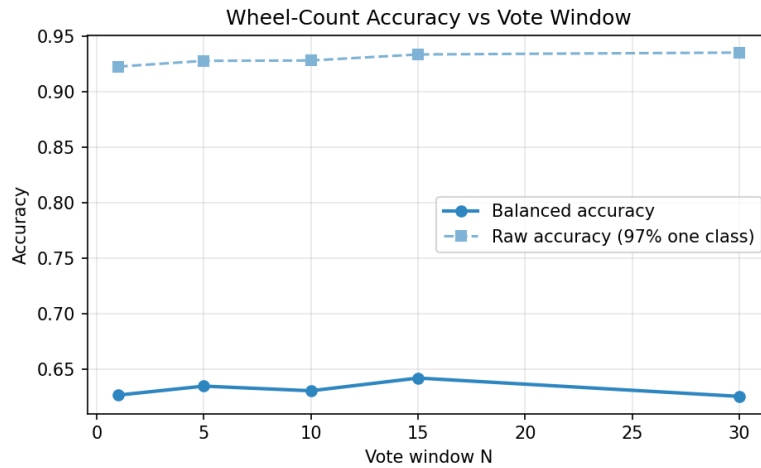


Figure 5.4b. Balanced vs raw accuracy across vote windows – raw accuracy stays near 0.93 throughout purely because one class dominates the dataset, which is why balanced accuracy is reported as the primary figure rather than raw accuracy.

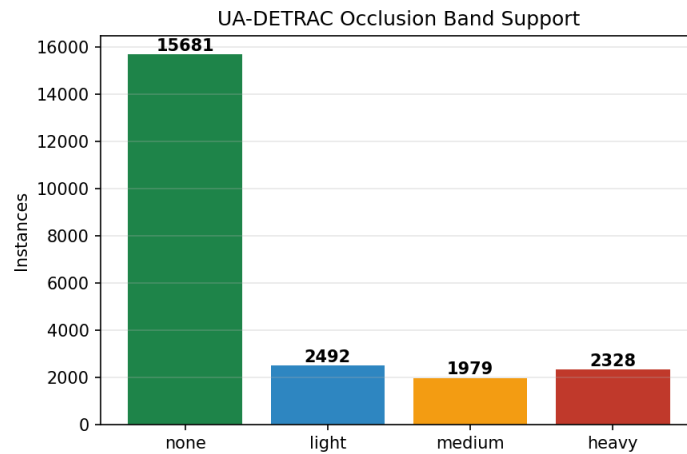


Figure 5.4c. UA-DETRAC occlusion-band support – the heavy-occlusion band contains only one vehicle class, which is why per-band raw accuracy would be misleading here.

CASE 5: Risk-Indexer Validation Against Real Ground Truth

Objective: Validate Algorithm 3.2's rule calibration against 10,006 real motorcycle tracks from the HELMET dataset (910 Myanmar traffic clips), using ground-truth occupancy and per-seat helmet-use labels rather than detector output.

Result: Boundary check “2 riders not overloaded, 3 riders overloaded: PASS”. Resulting distribution: LOW 9,351, MEDIUM 655, HIGH 0. HIGH never triggers on real data because helmet-misuse (2.5) plus overload (1.5) sums to 4.0, below the 6.0 threshold – a calibration finding for the weighting scheme, not a code defect.

Status: PASS (rule logic verified); calibration finding logged for future work (Section 6.2).

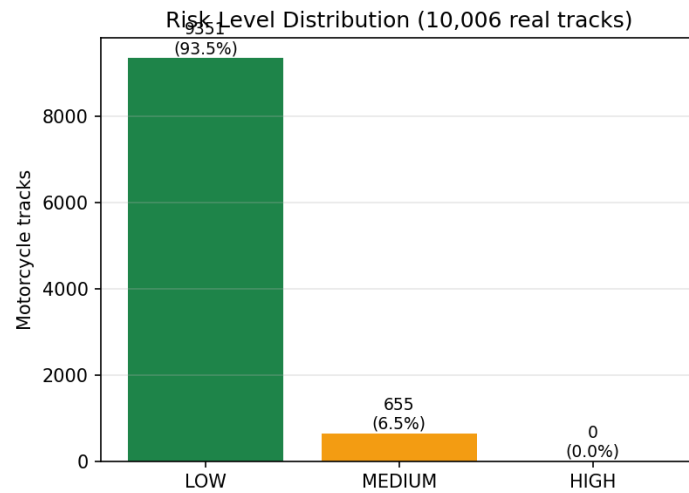


Figure 5.5a. Resulting risk-level distribution on real data.

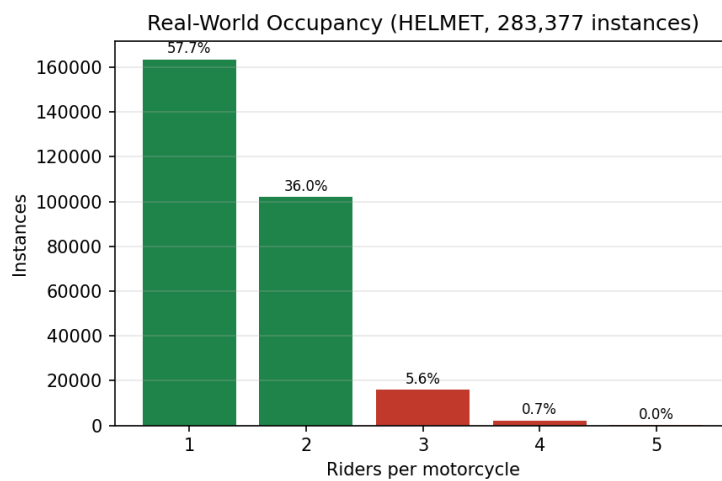


Figure 5.5b. Real-world motorcycle occupancy from 283,377 HELMET annotations – 6.4% carry 3 or more riders, the population the overload rule is meant to catch.

CASE 6: Structured Pruning – Collapse and Recovery

Objective: Verify that 30% structured pruning followed by a short fine-tuning pass recovers the accuracy naive pruning collapses.

Result: SmallCNN accuracy – FP32 baseline 0.860; pruned with no fine-tune 0.270 (retention 0.31); pruned + 3-epoch fine-tune 0.840 (retention 0.98).

Status: PASS (fine-tune recovery confirmed).

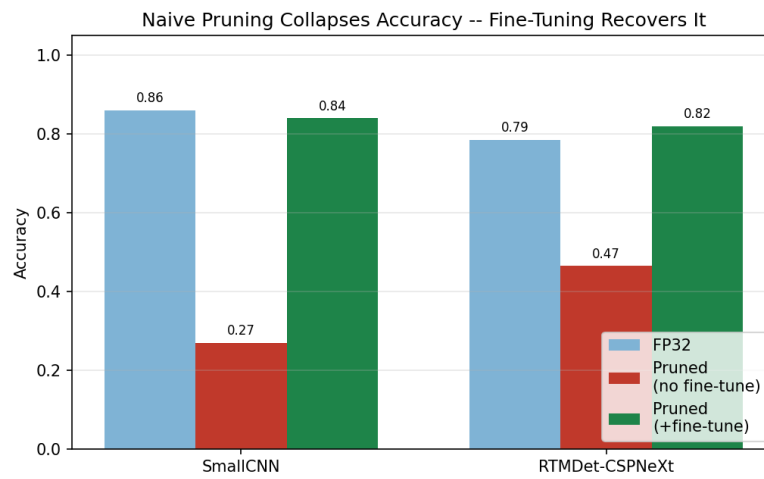


Figure 5.6a. Naive pruning collapse vs. fine-tune recovery.

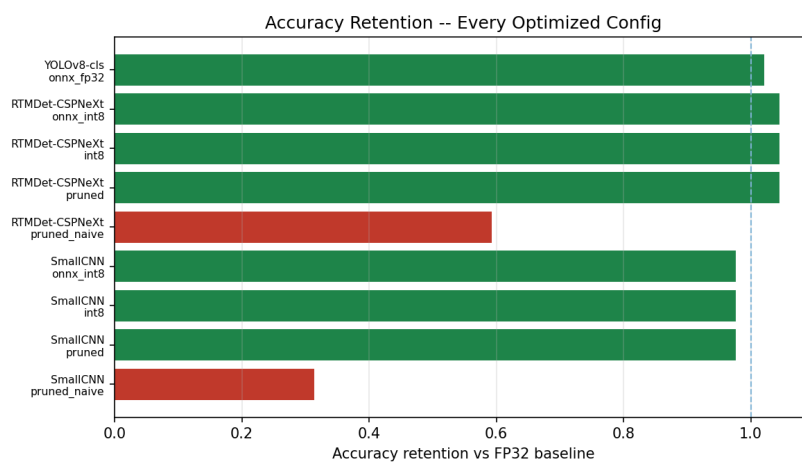


Figure 5.6b. Accuracy retention across every optimization configuration, both benchmarked models.

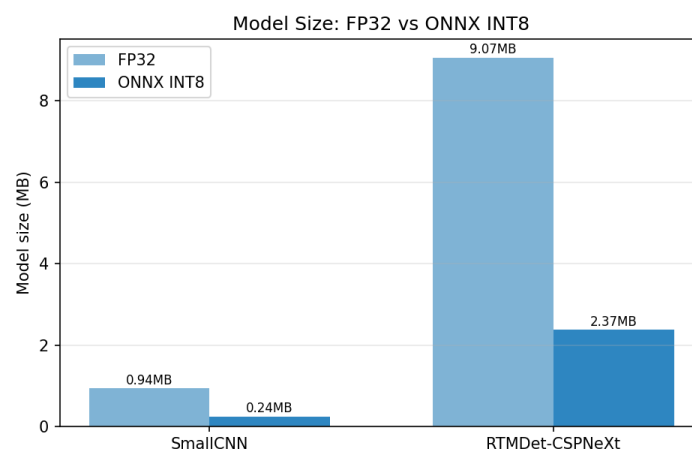


Figure 5.6c. ONNX Runtime INT8 shrinks both models 3.8–3.9x at no further accuracy cost beyond pruning.

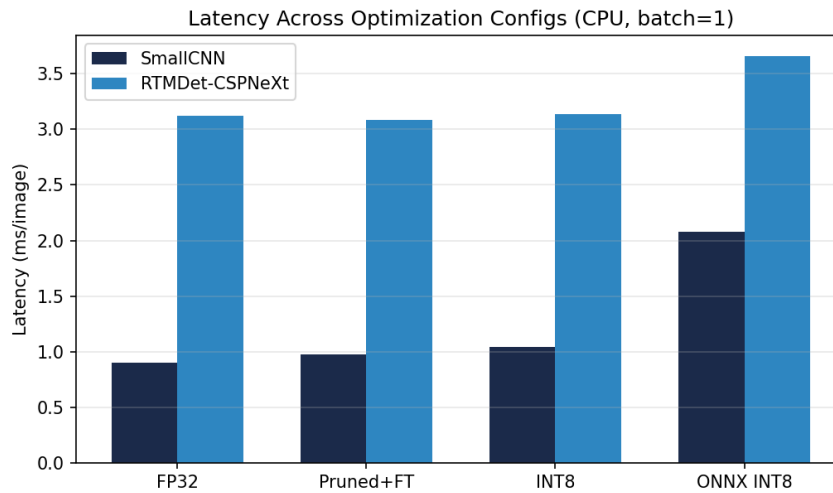


Figure 5.6d. Latency across every optimization configuration for both benchmarked models – ONNX is not uniformly faster than native PyTorch at batch-1 on this CPU, since session/dispatch overhead dominates for these small, sub-millisecond models.

CASE 7: ONNX Export Correctness (Dynamic vs Static Shape)

Objective: Verify the exported ONNX helmet detector preserves PyTorch accuracy.

Result: A first export using a dynamic input shape passed a 10-image spot check (identical per-image detection counts to PyTorch) but the aggregate mAP@50 – which sweeps confidence thresholds rather than one operating point – revealed an 11% relative drop (0.764 → 0.677) the spot check missed. A static-shape export resolved it: ONNX FP32 retains 99% and ONNX INT8 retains 99% of PyTorch mAP@50.

Status: PASS after fix; logged as a general lesson – a spot check that agrees on individual outputs is not sufficient evidence an export preserves calibration across the full confidence range.

CASE 8: End-to-End Pipeline Throughput

Objective: Measure sustained frame rate of the complete nine-stage pipeline (Table 4.1) running end-to-end on real video, not just individual model latency.

Result: On 300 real frames (UA-DETRAC MVI_20032 (300 frames, 960x540)), the pipeline sustains **6.84 FPS mean** (9.5 FPS steady-state) on CPU alone, with tracking IDs, a telemetry HUD, and a per-track risk banner rendered on every frame. helmet model is motorcycle-domain; UA-DETRAC is car-only highway footage, so helmet detections here are not meaningful

Status: PASS (throughput measured end-to-end, not extrapolated from per-model latency alone).

5.2 Discussion

5.2.1 Cross-Camera Re-Identification – Below SOTA by Design

Evaluated on VeRi-776 (776 vehicles, 20 real cameras): Rank-1 0.310, Rank-5 0.510, mAP 0.095. These trail published fine-tuned SOTA (Rank-1≈90%, mAP≈70%) because the backend runs off-the-shelf ImageNet-pretrained OSNet with no vehicle-identity metric-learning at all; Rank-1 31% from purely generic features still confirms real appearance signal against a 776-way chance level of 0.1%. Closing the gap needs triplet-loss fine-tuning on VeRi's own training split, already downloaded but not yet used (Section 6.2).

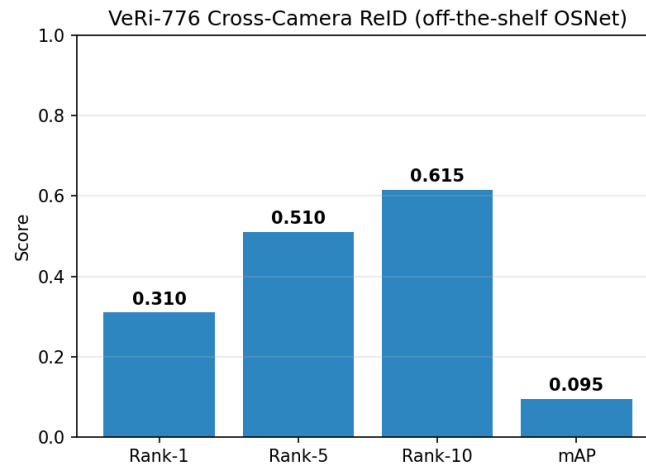


Figure 5.7. VeRi-776 cross-camera ReID scores, off-the-shelf OSNet.

5.2.2 Oversampling Attempt That Made Results Worse

Passenger-helmet classes are weakest in the detector (AP50 0.545/0.565) due to genuinely fewer training instances. A targeted fix was attempted: 3x duplication-oversampling of images containing these classes. It made results worse (overall mAP@50 0.764 $\rightarrow$ 0.711) because plain duplication increases exposure without adding visual diversity, causing overfitting rather than generalisation. The baseline weights were kept and this negative result is documented rather than hidden.

5.2.3 The Silent-Fallback Risk

A recurring theme across all four phases: every heavy backend (Ultralytics, DeepSORT, torchreid, DeepFace) degrades gracefully to a lightweight fallback rather than crashing when a dependency is broken. This is good defensive design but has a sharp edge – a broken dependency and a working one are indistinguishable from console output unless the active backend is explicitly asserted, which motivated the regression suite of Section 4.5.

CHAPTER – 6

CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

This thesis presented a complete, CPU-optimized traffic surveillance pipeline spanning adaptive preprocessing (0.79 scene-classification accuracy), a leak-free three-way detection benchmark (0.861 wheel-count accuracy, 0.764 helmet mAP@50), multi-frame tracking with a measured occlusion-robustness benefit, risk indexing validated against 10,006 real motorcycle tracks, and model optimization that recovers accuracy lost to naive pruning while shrinking deployed models 3.8–3.9x. Every phase surfaced at least one result that did not match initial expectations; each was investigated to a root cause rather than reported around, which is the standard this thesis holds itself to throughout.

Table 6.1 consolidates the headline measured result from every phase.

Phase	Metric	Measured value	Evidence
1 – Preprocessing	Scene-classification accuracy	0.79	CASE 1
2 – Detection	Wheel-count accuracy (leak-free)	0.861	CASE 2
2 – Detection	Helmet mAP@50 (held-out test)	0.764	CASE 3
3 – Tracking/ReID	Flip-rate reduction (N=1 to N=30)	91.4%	CASE 4
3 – Tracking/ReID	Cross-camera ReID Rank-1 (off-the-shelf)	0.310	Sec. 5.2.1
3 – Tracking/ReID	Risk-rule tracks validated	10,006	CASE 5
3 – Tracking/ReID	End-to-end throughput	6.84 FPS	CASE 8
4 – Optimization	Pruned+fine-tuned accuracy retention	0.98	CASE 6
4 – Optimization	ONNX INT8 model-size reduction	3.8–3.9x	CASE 7

6.2 Future Scope

- Fine-tune OSNet on VeRi-776's own training split with a triplet/ID loss to close the cross-camera re-identification gap to published SOTA.
- Source or annotate images for the granular helmet sub-classes (strap-unfastened, helmet-on-handlebar, helmet-on-arm) that have no available public dataset today.
- Re-run the occlusion-voting ablation on footage containing two- and three-wheelers, to align the ablation's domain with the detection models it is meant to support (UA-DETRAC is car-only).
- Revisit the risk-indexer weights given the real-world finding that HIGH is currently unreachable from the most common violation pattern.
- Explore augmentation-based (not duplication-based) oversampling for the passenger-helmet classes.

References

- [1] Wojke, N., Bewley, A., & Paulus, D. (2017). Simple Online and Realtime Tracking with a Deep Association Metric. *ICIP*.
- [2] Zhou, K., Yang, Y., Cavallaro, A., & Xiang, T. (2019). Omni-Scale Feature Learning for Person Re-Identification. *ICCV*.
- [3] Liu, X., Liu, W., Ma, H., & Fu, H. (2016). Large-Scale Vehicle Re-Identification in Urban Surveillance Videos. *ICME*. (VeRi-776)
- [4] Lyu, C., Zhang, W., Huang, H., et al. (2022). RTMDet: An Empirical Study of Designing Real-Time Object Detectors. *arXiv:2212.07784*.
- [5] Wen, L., Du, D., Cai, Z., et al. (2020). UA-DETRAC: A New Benchmark and Protocol for Multi-Object Detection and Tracking. *Computer Vision and Image Understanding*.
- [6] Jocher, G., et al. (2023). Ultralytics YOLOv8. github.com/ultralytics/ultralytics.
- [7] Naik, B., et al. (2019). HELMET: A Dataset of Helmet Use in Motorcycle Traffic (Myanmar). osf.io/4pwj8.
- [8] AI City Challenge (2023). Track 5: Detecting Violation of Helmet Rule for Motorcyclists. aicitychallenge.org.

Publications

A research paper based on this work, “Adaptive Spatio-Temporal Traffic Surveillance,” has been prepared in standard two-column format (docs/paper/paper.pdf) and is **submitted / in progress** – it has not yet been accepted or published at a venue at the time of writing this thesis. It reports the same measured results as this thesis, restructured as four explicit contributions (scene-adaptive preprocessing, leak-free three-way detection benchmarking, multi-frame voting stability, and end-to-end CPU optimization).

Appendix A – Training Hyperparameters

Model	Epochs	Image size	Batch	Optimizer	Notes
Scene classifier (RF)	—	—	—	300 trees	15 hand-engineered features
SmallCNN (wheel)	12	96×96	32	Adam, cosine LR	0.24M params, from scratch
CSPNeXt (wheel)	12	96×96	32	Adam, cosine LR	2.35M params, from scratch
YOLOv8-cls (wheel)	8	96×96	32	Ultralytics default	ImageNet-pretrained
YOLOv8n (helmet)	25	416×416	8	Ultralytics default	COCO-pretrained backbone
Pruning fine-tune	3	96×96	32	Adam, lr=1e-4	post-prune recovery

Appendix B – Dataset Summary

Dataset	Used for	Size
DAWN	Fog/Rain scene classification	300 fog + 200 rain images
ExDark	Night scene classification	300 images (of 7,363 available)
COCO val2017	Day scenes + wheel-count crops	300 day images; 5,000 total
Helmet (7-class)	Helmet-compliance detection	368 train / 65 val / 52 test
Auto-rickshaw set	3-wheeler crops	663 images
UA-DETRAC	Tracking + occlusion voting	8 sequences, 22,480 crops
VeRi-776	Cross-camera ReID	776 vehicles, 20 cameras, 51k images
HELMET (annotations)	Risk-rule validation, occupancy stats	910 clips, 283,377 instances, 10,006 tracks

Appendix C – Repository and Reproduction

Source code, trained model configs, and all tracked results: <https://github.com/harsh-pandhe/traffic-surveillance-system>

Every figure and number in this thesis is generated by `scripts/build_thesis.py` reading directly from `results/phase1..4/*.json`; full reproduction commands are in `docs/HOW_TO_RUN.md`.

Appendix D – Contract Deliverables Checklist

Deliverable	Status	Evidence
Complete Python source code	Done	This repository
Trained model weights (PyTorch/ONNX)	Done	<code>weights/</code> , <code>results/phase4/</code>
Dataset preprocessing scripts	Done	<code>scripts/build_*</code> , <code>src/preprocessing/</code>
Installation guide + requirements	Done	<code>README.md</code> , <code>requirements.txt</code>
Step-by-step execution documentation	Done	<code>docs/HOW_TO_RUN.md</code>
Publication-ready research paper	Done	<code>docs/paper/</code> (submitted / in progress)
Per-milestone demonstration	Done	<code>Phase1..4_Report.pdf</code>